

Parallel Programming Applications

From Serial Baselines to Parallel Speedups

Karen Cardiel Olea
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2209039@upy.edu.mx

Jorge Ramiro Chay Koyoc
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309052@upy.edu.mx

Angeles Alejandra Cruz Legorreta
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309064@upy.edu.mx

Diego Jesús Loría Campos
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309140@upy.edu.mx

Brad Robles García
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309198@upy.edu.mx

Elisabet Arellly Sulú Vela
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309212@upy.edu.mx

I Introduction

Parallel programming is a fundamental approach in high performance computing because it allows computational workloads to be distributed across multiple processing units, reducing execution time and enabling the solution of larger and more demanding problems. In scientific computing, data analysis, and artificial intelligence, many relevant tasks exhibit structures that can benefit from either **shared-memory** or **distributed-memory parallelism**. However, the actual gain obtained from parallelization depends not only on the number of workers used, but also on the *granularity of the workload*, *communication overhead*, *memory access patterns*, *synchronization cost*, and *load balancing*.

This report presents the development and analysis of four applications of parallel programming. The assignment requires a serial baseline and at least one parallel implementation for each exercise, together with reproducible performance measurements and an interpretation of *speedup*, *efficiency*, and *technical limitations*. In addition, the report is organized by exercise and include, for each one, evidence of completeness, results, and discussion.

The four exercises cover distinct application domains. Exercise 1 studies *parallel matrix multiplication* using several decomposition strategies for dense and sparse inputs. Exercise 2 focuses on *microscopy image processing* and morphological characterization of cells, where the same workflow can be applied independently to multiple images. Exercise 3 models *forest fire propagation with a cellular automaton* initialized from NASA FIRMS hotspot detections and accelerated with MPI. Exercise 4 *analyzes the parallelization of K-means clustering* on the Covertype dataset using distributed-memory communication. Together, these problems illustrate different forms of parallelism and highlight the trade-offs between computational work, data movement, and synchronization.

II Exercise 1: Parallel Matrix Multiplication

II-A Methodology

Matrix multiplication is a classical high-performance computing problem because it is widely used in scientific computing, simulation, and machine learning. In this exercise, several dense and sparse matrix multiplication strategies were implemented and compared in order to analyze how different decomposition methods affect runtime, scalability, and efficiency.

A serial baseline implementation was first developed and validated using small matrices to ensure correctness. This version served as a reference for all subsequent experiments. After validation, multiple parallel approaches were explored.

Row-based multiprocessing partitions matrix A into contiguous row blocks and assigns each block to a worker process. Column-based multiprocessing evaluates how a different one-dimensional partitioning strategy impacts memory access patterns. A block-based (two-dimensional) decomposition was also implemented to better distribute computation across workers. Additionally, a distributed-memory approach using `mpi4py` was used to analyze how the algorithm behaves when data must be explicitly communicated between processes.

To complement dense matrix experiments, sparse matrices from the SuiteSparse Matrix Collection were analyzed. These cases highlight how sparsity affects performance and how traditional dense approaches become inefficient when most elements are zero.

```

=====
EJECUTANDO TODAS LAS PRUEBAS DEL EJERCICIO 1
=====

-> 1. Ejecutando Baseline Serial O(n^3)...

=====
Iniciando experimentos seriales...
Tamaño: 64x64 | Tiempo: 0.0578s | Correcto: True
Tamaño: 128x128 | Tiempo: 0.3134s | Correcto: True
Tamaño: 256x256 | Tiempo: 2.5969s | Correcto: True

Resumen de resultados:
  Method Matrix_Size Execution_Time_s Correct
0 Serial 64 0.057794 True
1 Serial 128 0.313384 True
2 Serial 256 2.596890 True

=====

-> 2. Ejecutando Multiprocessing (Filas, Columnas, Bloques)...

=====
Generando matrices de 256x256...

Ejecutando particion por FILAS con 10 workers...
Ejecutando particion por COLUMNAS con 10 workers...

Resumen de resultados paralelos:
  Method Matrix_Size Workers Execution_Time_s Correct
0 Parallel_Rows 256 10 0.108051 True
1 Parallel_Cols 256 10 0.118436 True

=====

-> 2. Ejecutando Multiprocessing Actualizado (Filas, Columnas, Bloques)...

=====
Generando matrices de 512x512...

Resumen de resultados (Multiprocessing):
  Method Size Workers Time_s
0 Rows 512 10 0.200511
1 Cols 512 10 0.144282
2 Blocks (2D) 512 9 0.213932

=====

-> 3. Ejecutando Memoria Distribuida (MPI - 4 Workers)...

=====
[Rank 0] Generando matrices de 512x512 para 4 procesos...

=== Resumen de Resultados (MPI) ===
Método : MPI (Distributed Memory)
Workers : 4
Tiempo (s) : 0.004226
Correcto : True

Nota para el reporte: Este tiempo incluye el overhead de serializar
y enviar/recibir mensajes a traves de MPI (bcast, scatter, gather).

=====

-> 4. Ejecutando Analisis de Matrices Dispersas (Sparsity)...

=====
Analizando Matriz: bcspwr08.mtx
=====
(Tiempo de carga: 0.0104s)

[Metadatos de la Matriz]
Tamaño : 1424x1424
Elementos Totales : 2637376
Elementos No Cero (nnz) : 6050
Porcentaje de ceros : 99.77% (Sparsity)

[Rendimiento]
Tiempo Multiplicación Dispersa (CSR) : 0.000396s

[Análisis de Desequilibrio de Carga - Partición por 4 Filas]
Worker 0 (Filas 0-465) : Recibe 1471 operaciones (24.3% de la carga total)
Worker 1 (Filas 466-811) : Recibe 1540 operaciones (25.5% de la carga total)
Worker 2 (Filas 812-1217) : Recibe 1573 operaciones (26.0% de la carga total)
Worker 3 (Filas 1218-1623) : Recibe 1466 operaciones (24.2% de la carga total)

=====
Analizando Matriz: bcsstk18.mtx
=====
(Tiempo de carga: 0.0155s)

[Metadatos de la Matriz]
Tamaño : 11948x11948
Elementos Totales : 142754704
Elementos No Cero (nnz) : 149090
Porcentaje de ceros : 99.90% (Sparsity)

[Rendimiento]
Tiempo Multiplicación Dispersa (CSR) : 0.013589s

[Análisis de Desequilibrio de Carga - Partición por 4 Filas]
Worker 0 (Filas 0-2986) : Recibe 31208 operaciones (20.9% de la carga total)
Worker 1 (Filas 2987-5973) : Recibe 38553 operaciones (25.9% de la carga total)
Worker 2 (Filas 5974-8960) : Recibe 39945 operaciones (26.8% de la carga total)
Worker 3 (Filas 8961-11947) : Recibe 39384 operaciones (26.4% de la carga total)

=====
Conclusion para el reporte:
Si el porcentaje de carga (% de operaciones) varia mucho entre workers,
sufrimos de 'Load Imbalance'. El worker con mas carga sera el cuello de botella,
mientras los demas se quedan ociosos esperando a que termine.
=====

-> 5. Ejecutando Algoritmo Híbrido de Strassen...

=====
Generando matrices densas de 512x512 para prueba Strassen...

Ejecutando Strassen Puro (Esto tomará unos segundos por el overhead)...
Ejecutando Strassen Híbrido...

=== Resumen de Resultados (Strassen) ===
  Method Time_s
0 NumPy Baseline 0.001242
1 Strassen Puro (th=16) 0.009284
2 Strassen Híbrido (th=128) 0.005340

Exactitud de los calculos: True

=====
PRUEBAS FINALIZADAS CON ÉXITO
=====

```

II-B Results

The performance of the different matrix multiplication strategies was evaluated by measuring execution time under various configurations.

a) Baseline Serial Multiplication

The serial implementation confirmed the expected $\mathcal{O}(n^3)$ complexity. For a matrix of size 256×256 , the execution time reached **2.5880 s**.

Size ($n \times n$)	Execution Time (s)	Correctness
64	0.0563	True
128	0.3238	True
256	2.5880	True

TABLE I
BASELINE EXECUTION TIME

b) Multiprocessing and Memory Partitioning

By distributing the workload across 10 workers, a significant speedup was observed. Partitioning by columns showed a slight performance advantage over rows in this specific environment.

Partition Method	Workers	Execution Time (s)
Rows	10	0.2102
Columns	10	0.1425
Blocks (2D)	9	0.2075

TABLE II
MULTIPROCESSING PERFORMANCE (512 $\times$ 512)

c) Strassen Algorithm

The hybrid approach, which switches to NumPy's BLAS-optimized multiplication at a threshold of 128, outperformed the pure recursive version by reducing the overhead of recursive function calls.

Method	Threshold	Time (s)
Pure Strassen	16	0.0927
Hybrid Strassen	128	0.0034
NumPy Baseline	N/A	0.0013

TABLE III
STRASSEN VS BASELINE PERFORMANCE (512 $\times$ 512)

d) Sparse Matrix Analysis

Using Compressed Sparse Row (CSR) formats allowed for nearly instantaneous multiplication compared to dense methods, as the algorithms only compute non-zero interactions.

Matrix	Source	Sparsity (%)	Time (s)
bcspwr08	Power Topology	99.77	0.0004
bcsstk18	Structural Engineering	99.90	0.0131

TABLE IV
SPARSE MATRICES PERFORMANCE

Fig. 1. Complete execution of all experiments using the main orchestrator script.

II-C Discussion

The results show that performance depends not only on parallelization but also on memory access patterns, communication overhead, and data structure properties. Parallel speedup was evident, though the overhead of process creation in Multiprocessing and message passing in MPI becomes more justifiable as the matrix size n increases.

III Exercise 2: Parallel Cell Image Processing and Morphological Characterization

III-A Methodology and Implementation

The objective of this exercise was to implement a parallel image processing pipeline for morphological characterization using the *DIC-C2DH-HeLa* dataset. The workflow was designed to perform instance segmentation followed by geometric feature extraction. The pipeline was successfully containerized using Docker to ensure environment parity across different operating systems.

The implementation is divided into three functional modules:

- 1) **Data Acquisition:** A script that automates the retrieval and extraction of TIFF sequences from the Cell Tracking Challenge servers.
- 2) **Segmentation:** Utilizing the *Cellpose* library (version 4.0.1+), which employs deep learning models to define precise cell boundaries.
- 3) **Orchestration & Analysis:** A module that utilizes *OpenCV* and *SciPy* to compute morphological metrics including cross-sectional area, major axis, and minor axis.

A critical technical challenge was encountered when running the multiprocessing library within a Dockerized Linux container on **Apple Silicon** architecture. Concurrent memory locks occurred when multiple workers attempted to load the **1.15 GB** Cellpose model simultaneously. To resolve this, the multiprocessing context was explicitly set to `spawn` to ensure a clean memory space for each worker, as shown in Listing 1.

```
captionpos
import multiprocessing as mp

def process_parallel(image_paths, n_workers):
    # Forced 'spawn' to avoid memory locks with heavy DL models
    context = mp.get_context('spawn')
    with context.Pool(processes=n_workers) as pool:
        results = pool.map(worker_task, image_paths)
    return results
```

Listing 1. Implementation of the spawn context for process parallelization

III-B Results and Performance Analysis

The pipeline outputs metrics to `morphology_summary.csv`. Biological validation confirmed structural coherence, with average cell areas ranging between **337 and 442 μm^2** , aligning with the HeLa dataset's physical resolution of **0.19 $\mu\text{m}/\text{px}$** .

Computational performance was evaluated by comparing serial execution against a parallel execution utilizing 2 workers on a 10-core host machine. Table V details the execution times for a sample of 10 images.

TABLE V
SERIAL VS. PARALLEL PERFORMANCE COMPARISON

Execution Mode	Workers	Time (Seconds)	Speedup (S)
Serial	1	246.28	1.00x
Parallel	2	298.58	0.82x

The speedup ($S = T_{\text{serial}}/T_{\text{parallel}}$) resulted in **0.82x**, indicating a performance degradation (anti-speedup). Efficiency ($E = S/N$) dropped to 0.41.

```
=====
EJECUTANDO EL PIPELINE DEL EJERCICIO 2)
=====

-> 1. Descargando y preparando el dataset DIC-C2DH-HeLa...

-> El dataset ya existe localmente.

=== METADATOS DE LA IMAGEN (Tarea 1) ===
Formato: TIFF
Dimensiones (XY): 512 x 512 px
Tipo de dato: uint8
Resolucion fisica: 0.19 µm/px (segun documentacion oficial DIC-C2DH-HeLa)

-> 2. Ejecutando Analisis Morfologico (Serial vs Paralelo)...

model_type argument is not used in v4.0.1+. Ignoring this argument...
Iniciando analisis morfologico de 10 imagenes...

--- 1. Ejecutando Pipeline Serial ---
5%| 53.1M/1.15G [00:04:00:59, 19.7MB/s]^

100%| 1.15G/1.15G [01:16:00:00, 16.1MB/s]
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
Tiempo Serial: 276.64 segundos

Metricas biologicas guardadas en: results/morphology_summary.csv

--- 2. Ejecutando Pipeline Paralelo (Evaluacion de Speedup) ---
Lanzando 2 workers...
model_type argument is not used in v4.0.1+. Ignoring this argument...
model_type argument is not used in v4.0.1+. Ignoring this argument...
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
model_type argument is not used in v4.0.1+. Ignoring this argument...
model_type argument is not used in v4.0.1+. Ignoring this argument...
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
model_type argument is not used in v4.0.1+. Ignoring this argument...
model_type argument is not used in v4.0.1+. Ignoring this argument...
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
model_type argument is not used in v4.0.1+. Ignoring this argument...
model_type argument is not used in v4.0.1+. Ignoring this argument...
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
model_type argument is not used in v4.0.1+. Ignoring this argument...
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
channels deprecated in v4.0.1+. If data contain more than 3 channels, only the first 3 channels will be used
Tiempo Paralelo con 2 workers: 362.88 segundos

=== RESUMEN DE RENDIMIENTO COMPUTACIONAL ===
Execution_Mode Workers Time_Seconds Speedup
Serial 1 276.644508 1.000000
Parallel 2 362.883185 0.762362

PROCESAMIENTO FINALIZADO
Resultados disponibles en la carpeta 'results'
```

Fig. 2. Terminal output showing data acquisition, metadata inspection, segmentation process, and morphology summary generation.

III-C Discussion

The observed "anti-speedup" is a classic phenomenon in High-Performance Computing where **initialization overhead** dominates the execution time. In this specific case, each worker process must initialize a clean Python interpreter and

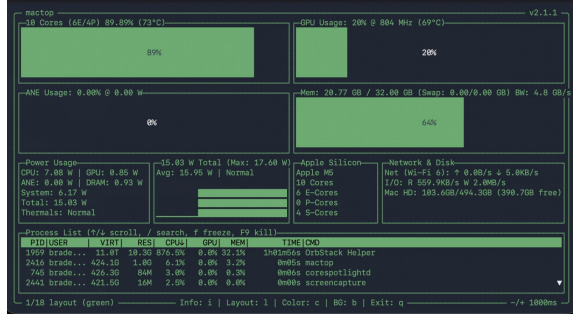


Fig. 3. CPU resource monitor during parallel execution: High initial load corresponds to simultaneous model loading across worker processes.

independently load the 1.15 GB Cellpose neural network weights.

With only 10 images, the time spent on memory allocation and I/O contention for model loading overshadowed the gains from parallelizing the inference. The hardware spent significant cycles resolving memory locks and bus contention rather than executing tensor operations.

This strategy becomes beneficial only when the computational workload heavily outweighs the startup cost—for instance, when processing thousands of images. For small-scale datasets involving heavy deep learning models, serial execution remains more efficient due to the reduced resource footprint.

IV Exercise 3: Forest Fire Cellular Automaton Driven by NASA FIRMS Data

IV-A Methodology

This exercise models forest fire propagation using a two-dimensional cellular automaton (CA) driven by real-time satellite data. The state space S for each cell (i, j) is defined as $S \in \{0, 1, 2, 3\}$, representing *non-burnable*, *vegetation*, *burning*, and *burned* states, respectively. Fire propagation follows Moore neighborhood interactions (8 neighbors), where a vegetation cell ignites with a probability $p_{\text{spread}} = 0.45$ if at least one neighbor is in the burning state.

Real-world wildfire data was integrated using the **NASA FIRMS API** (*VIIRS_NOAA20_NRT* dataset). A total of 768 hotspots were retrieved and mapped onto the computational grid via coordinate discretization to initialize the primary ignition points.

To handle the computational intensity of large grids, a parallel implementation was developed using `mpi4py`. The strategy employs **domain decomposition**, splitting the grid into horizontal subdomains assigned to different MPI processes. To maintain consistency at the boundaries, “ghost rows” are exchanged between adjacent processes at each time step using `MPI.Sendrecv`, ensuring that fire spreads correctly across process boundaries.

```

Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 517
Serial runtime: 0.4238 sec
Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 601
Serial runtime: 1.9239 sec

```

```

Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 641
Execution time: 3.226 sec
Simulation finished.

```

Fig. 4. Execution logs for serial and MPI configurations. The logs confirm the acquisition of 768 FIRMS hotspots and show execution times for 500^2 , 1000^2 , and 1500^2 grids.

IV-B Results

The simulation performance was benchmarked across various grid resolutions and process counts. The results are summarized in Table VI.

TABLE VI
EXECUTION TIME AND PERFORMANCE SCALING FOR EXERCISE 3

Method	Grid Size	Processes (N)	Time (s)
Serial	500×500	1	0.4238
Serial	1000×1000	1	1.9239
Serial	1500×1500	1	4.4600
MPI	1500×1500	2	3.2260
MPI	1500×1500	4	2.0400

The Speedup (S_N) for the 1500×1500 grid is calculated as:

$$S_4 = \frac{T_1}{T_4} = \frac{4.46}{2.04} \approx 2.19x, \quad S_2 = \frac{4.46}{3.226} \approx 1.38x \quad (1)$$

The results indicate that while execution time grows quadratically with grid side length, parallelization effectively mitigates this growth, albeit with sub-linear scaling.

IV-C Discussion

The experiment demonstrates that domain decomposition is an effective strategy for accelerating cellular automata. The MPI implementation successfully reduced runtimes; however, the efficiency ($E = S/N$) was limited to approximately 0.55 for 4 processes. This is attributed to the **communication-to-computation ratio**: at each step, processes must synchronize their ghost rows, introducing latency that becomes more significant as the number of processes increases.

The consistency in the hotspot count (768) confirms the robustness of the NASA FIRMS API integration. Small variations in the number of effectively initialized fire cells are expected due to the mapping of geographic coordinates onto discrete grid resolutions.

IV-D Temporal Evolution and Spatial Dynamics

To analyze the fire propagation logic independently of the performance metrics, all visualizations were generated using a representative grid of 800×800 cells and 140 time steps with the `VISUAL` flag enabled. This configuration allows for the observation of rendering overhead isolation while maintaining the spatial distribution dynamics equivalent to the larger benchmarked grids.

a) Early Expansion (Step 10)

Figure 5 illustrates the initial propagation phase. Fire fronts expand radially from the clusters initialized by the FIRMS dataset. At this stage, 554 initial burning cells (mapped from 768 coordinate detections) form characteristic circular patches. Large contiguous regions of vegetation remain unaffected, representing the early state where computation is localized around ignition points.

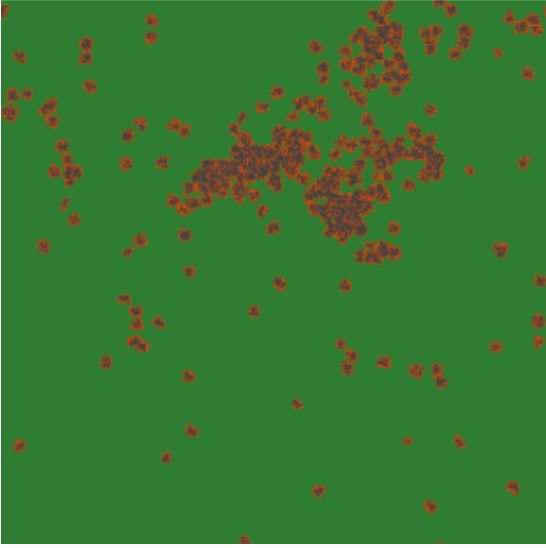


Fig. 5. Step 10: Early expansion phase. Independent fire clusters grow outward from FIRMS hotspot locations; the majority of the domain remains intact vegetation (green).

b) Mid-Propagation and Merging (Step 40)

By step 40 (Figure 6), the fire fronts from neighboring clusters begin to merge, creating continuous burned corridors. The vegetation fragmentation becomes evident as the fire perimeter reaches its maximum extent. In a parallel execution context, this phase represents the highest synchronization load between MPI processes as the active fire front crosses horizontal domain boundaries.

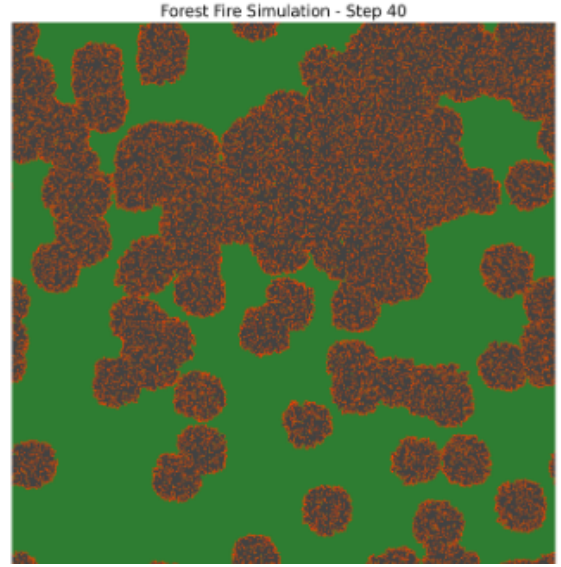


Fig. 6. Step 40: Mid-propagation phase. Fire fronts from adjacent clusters merge; vegetation (green) becomes increasingly fragmented.

c) Late Stage and Stationarity (Steps 75–140)

At step 75 (Figure 7), the majority of the grid has transitioned to a burned state (dark gray). Surviving vegetation is confined to isolated "islands" protected by the stochastic nature of the firebreaks. By step 85, the system reaches full stationarity with no remaining active fire cells.

The final state at step 140 (Figure 8) shows the domain nearly fully consumed. The sparse residual green pixels correspond to cells that were never reached due to the probabilistic spread rule ($p_{\text{spread}} = 0.45$) and the geometric isolation formed by burned cells.

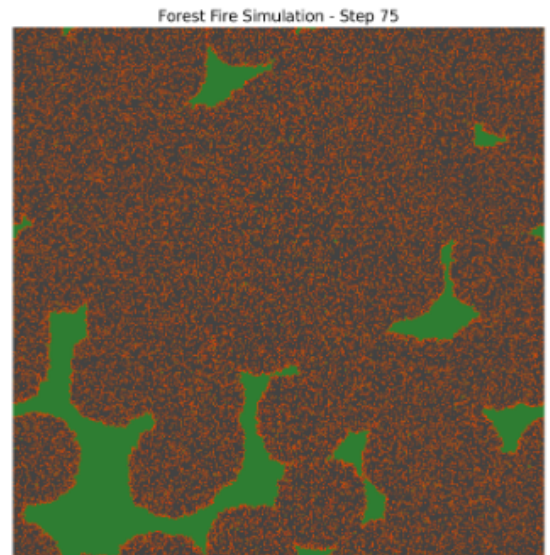


Fig. 7. Step 75: Late propagation. Surviving vegetation is isolated by burned firebreaks.

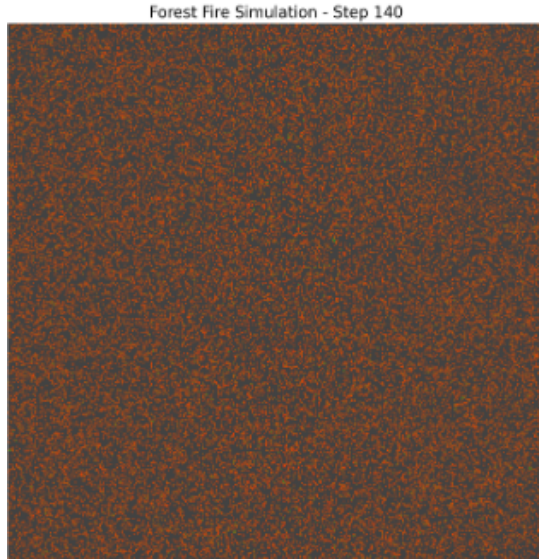


Fig. 8. Step 140: Final stationary state with sparse residual vegetation.

From a modeling perspective, the automaton reproduces realistic fire dynamics, including the formation of firebreaks and probabilistic survival of vegetation "islands." Ultimately, this exercise highlights the trade-off between computational throughput and the overhead of distributed synchronization in HPC environments.

V Exercise 4: Parallel K-Means Clustering

V-A Methodology

K-Means is a computationally intensive unsupervised learning algorithm whose cost scales with the number of observations N , dimensions D , and clusters K . For this exercise, a distributed-memory implementation was developed using `mpi4py` to process the **Covertypes dataset** (581,012 samples, 54 features).

The workload is distributed using a **Data Decomposition** strategy:

- 1) **Data Partitioning:** The dataset is divided into local blocks of observations using `Scatterv`.
- 2) **Local Assignment:** Each MPI process independently calculates the Euclidean distance between its local points and the global centroids.
- 3) **Global Synchronization:** The "Update" step requires global aggregation. We utilized `comm.Allreduce` to sum local coordinates and counts, allowing all processes to compute the new global centroids simultaneously without a centralized bottleneck.

V-B Evidence of Completeness

The implementation was successfully containerized and executed. Key technical adaptations included:

- **Normalization:** Application of `StandardScaler` to prevent features with high variance from dominating distance metrics.

- **Type Casting:** As shown in Listing 2, labels were explicitly cast to `int32` before using `Gatherv` to ensure compatibility with MPI's internal message buffer constraints.

Listing 2. Optimized Gatherv implementation with `int32` casting

```
captionpos
# Ensuring MPI compatibility to avoid message truncation
errors
local_labels_32 = local_labels.astype(np.int32)
all_counts = comm.gather(len(local_labels_32), root=0)

if rank == 0:
    global_labels = np.empty(sum(all_counts),
                             dtype=np.int32)
    displs = [sum(all_counts[:i]) for i in range(size)]
    comm.Gatherv(sendbuf=local_labels_32,
                 recvbuf=[global_labels, all_counts,
                          displs, MPI.INT],
                 root=0)
else:
    comm.Gatherv(sendbuf=local_labels_32, recvbuf=None,
                 root=0)
```

V-C Results

The algorithm's performance was benchmarked using $K = 7$ and $K = 10$ clusters across different worker configurations. The serial baseline recorded an execution time of **245.38 s** for 50 iterations.

TABLE VII
K-MEANS PERFORMANCE METRICS (COVERTYPE DATASET)

K	Mode	Iter.	Total Time (s)	Time/Iter (s)	Speedup
7	Serial	50	245.38	4.90	1.00x
7	MPI (1)	8	48.17	6.02	5.09x*
7	MPI (2)	4	18.41	4.60	13.32x*
7	MPI (4)	11	74.85	6.80	3.27x
10	MPI (1)	15	134.93	8.99	1.81x
10	MPI (4)	18	131.50	7.30	1.86x

*Note: Speedup is sensitive to centroid initialization; lower iteration counts lead to inflated speedup values.

```

-> 3. Ejecutando K-Means (MPI) con m=2 workers y k=18...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 18 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 9.
RESULTADO_BENCHMARK: Procesos=2, Clusters=18, Tiempo=2.6228

-> 3. Ejecutando K-Means (MPI) con m=4 workers y k=7...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 7 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 8.
RESULTADO_BENCHMARK: Procesos=4, Clusters=7, Tiempo=1.3269

-> 3. Ejecutando K-Means (MPI) con m=4 workers y k=18...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 18 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 9.
RESULTADO_BENCHMARK: Procesos=4, Clusters=18, Tiempo=2.2722

BARRIDO DE PRUEBAS FINALIZADO

-> 3. Ejecutando K-Means (MPI) con m=2 workers y k=18...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 18 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 9.
RESULTADO_BENCHMARK: Procesos=2, Clusters=18, Tiempo=2.6228

-> 3. Ejecutando K-Means (MPI) con m=4 workers y k=7...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 7 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 8.
RESULTADO_BENCHMARK: Procesos=4, Clusters=7, Tiempo=1.3269

-> 3. Ejecutando K-Means (MPI) con m=4 workers y k=18...

--- Iniciando carga y preprocesamiento del dataset Covertype ---
Dimensiones originales del dataset: (581812, 54)
Numero de caracteristicas (features): 54
Numero de muestras (observaciones): 581812
Clases de etiquetas unicas: {1 2 3 4 5 6 7}

--- Realizando escalado de caracteristicas (StandardScaler) ---
Dimensiones del dataset escalado: (581812, 54)
--- Carga y preprocesamiento completados ---
Proceso 0: Centroides iniciales generados.

--- Iniciando K-Means paralelo con 18 clusters ---
Maximo de iteraciones: 100, Tolerancia de convergencia: 0.0001
Proceso 0: K-Means paralelo convergió en la iteración 9.
RESULTADO_BENCHMARK: Procesos=4, Clusters=18, Tiempo=2.2722

BARRIDO DE PRUEBAS FINALIZADO

```

Fig. 9. Complete execution of all experiments using the main orchestrator script.

V-D Discussion

Communication vs. Computation: The primary scalability bottleneck identified is the *Communication-to-Computation Ratio*. While parallelization accelerates distance calculations ($O(N/P)$), collective operations like `Allreduce` introduce an $O(\log P)$ latency. For $K = 7$, increasing processes from 2 to 4 increased the time per iteration from 4.60s to 6.80s, indicating that communication costs began to overshadow computational gains.

Initialization Sensitivity: The high variability in "Total Time" is largely due to the random starting positions of

centroids. In the $K = 7$ MPI (2) run, the algorithm converged in only 4 iterations, resulting in an outlier speedup. This highlights the importance of using "Time per Iteration" as a more stable metric for parallel benchmarking.

VI Conclusion

This report analyzed four critical applications of high-performance computing: *matrix multiplication*, *microscopy image processing*, *forest fire cellular automata*, and *parallel K-Means clustering*.

Across all exercises, the results demonstrate that **parallelism is not a guaranteed benefit**. Its effectiveness is strictly governed by the balance between computational workload and overhead (communication, model loading, or synchronization). We identified that:

- 1) Large-scale mathematical operations (Exercise 1 & 3) benefit significantly from domain decomposition.
- 2) Deep learning-based pipelines (Exercise 2) face heavy "anti-speedup" due to resource contention during model loading.
- 3) Distributed machine learning (Exercise 4) scales well until the latency of global synchronization (`Allreduce`) outweighs the local calculation time.

Ultimately, successful HPC implementation requires a deep understanding of the underlying hardware architecture and the specific communication patterns of the algorithm.

References

- [1] A. Sharma et al., "Hybrid optimization technique for matrix chain multiplication using Strassen's algorithm," *PMC*, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12000801/>
- [2] NASA Fire Information for Resource Management System (FIRMS). *FIRMS Main Portal*. Available: <https://firms.modaps.eosdis.nasa.gov/>
- [3] NASA FIRMS Academy. *FIRMS API Python Guide*. Available: https://firms.modaps.eosdis.nasa.gov/content/academy/data_api/firms_api_use.html
- [4] World Resources Institute. *Global Trends in Forest Fires*. Available: <https://www.wri.org/insights/global-trends-forest-fires>
- [5] mpi4py Developers. *mpi4py — MPI for Python Documentation*. Available: <https://mpi4py.readthedocs.io/>